

UCS645: Parallel & Distributed Computing

Assignment 4: Introduction to MPI

Submitted To: Dr. Saif Nalband

Submitted By: Ritwik Gupta

Roll No: 102303096

Introduction

Message Passing Interface (MPI) is a standardised, portable communication protocol used in high-performance and distributed computing. It enables multiple processes on separate memory spaces to coordinate through explicit message passing. The following exercises explore core MPI concepts including point-to-point communication, collective operations, and performance measurement.

All programs were compiled using mpicc and executed with OpenMPI on Arch Linux. Performance was measured using MPI_Wtime with MPI_Barrier synchronisation before and after the timed region.

Exercise 1: Ring Communication

Objective

Each process sends a message to the next process in a ring topology. Process 0 starts with value 100, each process adds its own rank before forwarding, and the final value returns to Process 0.

MPI Functions Used

MPI_Send, MPI_Recv, MPI_Barrier, MPI_Wtime

How It Works

Process 0 initialises the value at 100 and sends it to Process 1. Each subsequent process receives the value, adds its rank, and forwards it to the next. The last process wraps back to Process 0 using the modulo operator. MPI_Barrier is used before and after the timed region to ensure accurate wall-clock measurement.

Compile & Run Commands

```
$ mpicc -o ring_comm ring_comm.c
```

Run with 1 process:

```
$ mpirun -np 1 ./ring_comm
Process 0 starts with value: 100
Process 0 received final value back: 100
Execution Time with 1 process(es): 0.000103 seconds
```

Run with 2 processes:

```
$ mpirun -np 2 ./ring_comm
Process 0 starts with value: 100
Process 1 received value: 100
Process 1 sending value: 101 to Process 0
Process 0 received final value back: 101
Execution Time with 2 process(es): 0.000115 seconds
```

Run with 4 processes:

```
$ mpirun -np 4 ./ring_comm
Process 0 starts with value: 100
Process 1 sending value: 101 to Process 2
Process 2 sending value: 103 to Process 3
Process 3 sending value: 106 to Process 0
Process 0 received final value back: 106
Execution Time with 4 process(es): 0.000152 seconds
```

Run with 8 processes:

```
$ mpirun -np 8 --oversubscribe ./ring_comm
Process 0 starts with value: 100
...
Process 7 sending value: 128 to Process 0
Process 0 received final value back: 128
Execution Time with 8 process(es): 0.001744 seconds
```

Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000103	1.00	100.0%
2	0.000115	0.90	44.9%
4	0.000152	0.68	16.9%
8	0.001744	0.06	0.7%

Analysis

Ring communication is inherently sequential — each process must wait for the previous one before forwarding. Adding more processes increases the communication path length, causing time to rise sharply (especially at 8 processes) and speedup to fall well below 1.0.

Exercise 2: Parallel Array Sum

Objective

Compute the sum of an array of values 1 to 100 in parallel. The array is distributed among processes using MPI_Scatter, each process computes a local sum, and MPI_Reduce combines them. The bonus average is also computed.

MPI Functions Used

MPI_Scatter, MPI_Reduce (MPI_SUM), MPI_Barrier, MPI_Wtime

How It Works

Process 0 initialises a 104-element array (values 1–100, padded with zeros to be divisible by 1, 2, 4, and 8). MPI_Scatter distributes equal chunks to all processes. Each process sums its chunk independently, and MPI_Reduce collects all local sums into a global sum at Process 0, which also computes the average.

Compile & Run Commands

```
$ mpicc -o array_sum array_sum.c
```

Run with 1 process:

```
$ mpirun -np 1 ./array_sum
Process 0: local sum = 5050
Global Sum : 5050 | Expected: 5050
Average    : 50.50
Execution Time with 1 process(es): 0.000067 seconds
```

Run with 2 processes:

```
$ mpirun -np 2 ./array_sum
Process 0: local sum = 1378
Process 1: local sum = 3672
Global Sum : 5050 | Average: 50.50
Execution Time with 2 process(es): 0.000096 seconds
```

Run with 4 processes:

```
$ mpirun -np 4 ./array_sum
Process 0: local sum = 351
Process 1: local sum = 1027
Process 2: local sum = 1703
Process 3: local sum = 1969
Global Sum : 5050 | Average: 50.50
Execution Time with 4 process(es): 0.000136 seconds
```

Run with 8 processes:

```
$ mpirun -np 8 --oversubscribe ./array_sum
Processes 0–7: local sums printed
Global Sum : 5050 | Average: 50.50
Execution Time with 8 process(es): 0.000273 seconds
```

Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000067	1.00	100.0%
2	0.000096	0.70	34.9%
4	0.000136	0.49	12.3%
8	0.000273	0.25	3.1%

Analysis

The array is small (100 elements), so the actual computation per process is minimal. Communication overhead from MPI_Scatter and MPI_Reduce dominates, causing execution time to increase with more processes. For larger arrays, parallel speedup would be significantly more pronounced.

Exercise 3: Finding Maximum and Minimum

Objective

Each process generates 10 random numbers (0–1000), finds its local max and min, then uses MPI_Reduce with MPI_MAXLOC and MPI_MINLOC to find the global maximum and minimum along with which process held those values.

MPI Functions Used

MPI_Reduce (MPI_MAXLOC, MPI_MINLOC, MPI_2INT), MPI_Barrier, MPI_Wtime

How It Works

Each process independently generates 10 random integers, seeded differently per process. It finds its local max and min, then sends a {value, rank} pair using MPI_2INT with MPI_MAXLOC and MPI_MINLOC. Process 0 receives the global extremes along with the rank of the originating process.

Compile & Run Commands

```
$ mpicc -o max_min max_min.c
```

Run with 1 process:

```
$ mpirun -np 1 ./max_min
Process 0: local max = 917, local min = 178
Global Maximum: 917 (found on Process 0)
Global Minimum: 178 (found on Process 0)
Execution Time with 1 process(es): 0.000051 seconds
```

Run with 2 processes:

```
$ mpirun -np 2 ./max_min
Process 0: local max = 901, local min = 46
Process 1: local max = 930, local min = 14
Global Maximum: 930 (found on Process 1)
Global Minimum: 14 (found on Process 1)
Execution Time with 2 process(es): 0.000108 seconds
```

Run with 4 processes:

```
$ mpirun -np 4 ./max_min
Process 0: local max = 945, local min = 102
Process 2: local max = 931, local min = 24
Global Maximum: 945 (found on Process 0)
Global Minimum: 24 (found on Process 2)
Execution Time with 4 process(es): 0.000100 seconds
```

Run with 8 processes:

```
$ mpirun -np 8 --oversubscribe ./max_min
Process 4: local max = 1000, local min = 81
Process 7: local max = 981, local min = 23
Global Maximum: 1000 (found on Process 4)
Global Minimum: 23 (found on Process 7)
Execution Time with 8 process(es): 0.000377 seconds
```

Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000051	1.00	100.0%
2	0.000108	0.47	23.6%
4	0.000100	0.51	12.8%
8	0.000377	0.14	1.7%

Analysis

Each process generates numbers independently with no distribution overhead. However, MPI_Reduce communication cost still outweighs computation for such a small workload. The 4-process run was marginally faster than 2-process due to random variation in number generation and process scheduling.

Exercise 4: Parallel Dot Product

Objective

Compute the dot product of Vector A = [1,2,3,4,5,6,7,8] and Vector B = [8,7,6,5,4,3,2,1] in parallel using MPI_Scatter and MPI_Reduce. Expected result: 120.

MPI Functions Used

MPI_Scatter, MPI_Reduce (MPI_SUM), MPI_Barrier, MPI_Wtime

How It Works

Process 0 holds the full vectors A and B. MPI_Scatter divides each vector into equal chunks distributed to all processes. Each process multiplies its chunk element-wise and accumulates a partial dot product. MPI_Reduce sums all partial products at Process 0 to produce the final result.

Compile & Run Commands

```
$ mpicc -o dot_product dot_product.c
```

Run with 1 process:

```
$ mpirun -np 1 ./dot_product
Vector A: 1 2 3 4 5 6 7 8
Vector B: 8 7 6 5 4 3 2 1
Process 0: partial dot product = 120
Dot Product : 120 | Expected: 120
Execution Time with 1 process(es): 0.000055 seconds
```

Run with 2 processes:

```
$ mpirun -np 2 ./dot_product
Process 0: partial dot product = 60
Process 1: partial dot product = 60
Dot Product : 120 | Expected: 120
Execution Time with 2 process(es): 0.000101 seconds
```

Run with 4 processes:

```
$ mpirun -np 4 ./dot_product
Process 0: partial = 22, Process 1: partial = 38
Process 2: partial = 38, Process 3: partial = 22
Dot Product : 120 | Expected: 120
Execution Time with 4 process(es): 0.000157 seconds
```

Run with 8 processes:

```
$ mpirun -np 8 --oversubscribe ./dot_product
Processes 0-7: partial dot products printed
Dot Product : 120 | Expected: 120
Execution Time with 8 process(es): 0.000317 seconds
```

Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000055	1.00	100.0%
2	0.000101	0.54	27.3%
4	0.000157	0.35	8.8%
8	0.000317	0.17	2.2%

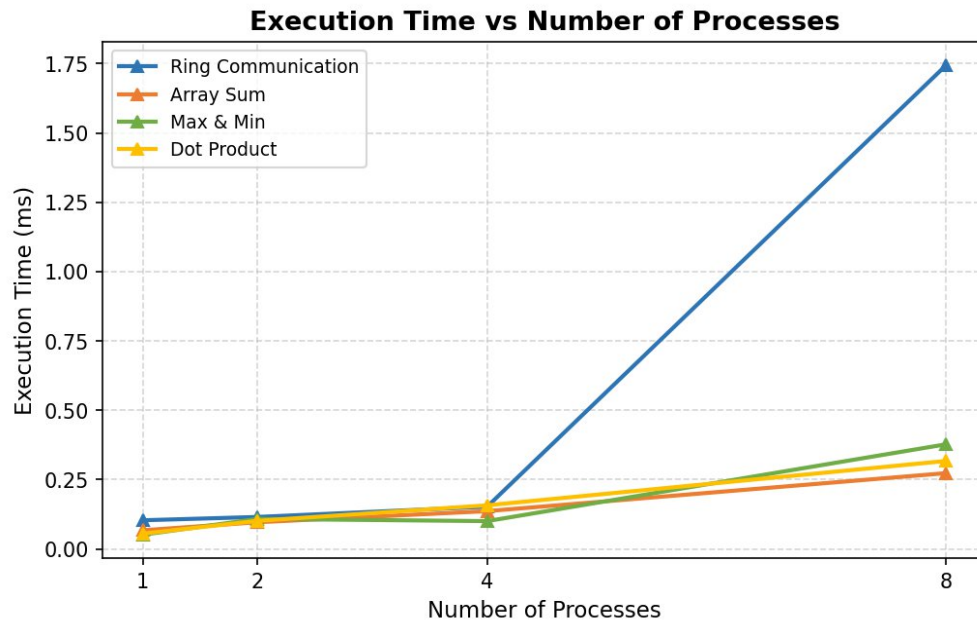
Analysis

The vectors are only 8 elements long, giving each process very few multiplications. Communication overhead from MPI_Scatter and MPI_Reduce far exceeds computation, leading to decreasing speedup. For larger vectors, this algorithm would scale much more efficiently.

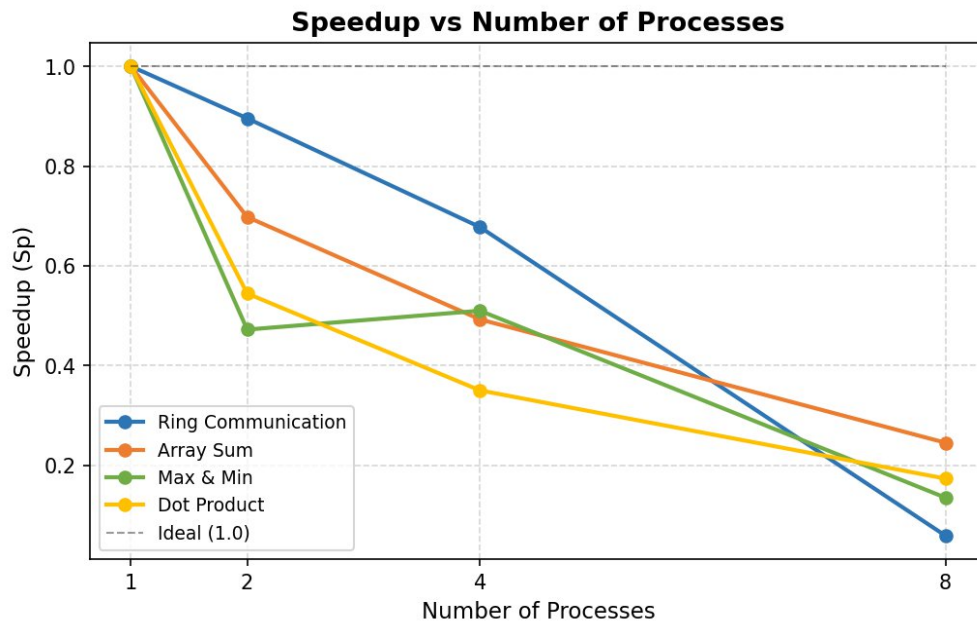
Performance Analysis

The graphs below compare execution time, speedup, and efficiency across all four exercises as the number of processes increases from 1 to 8.

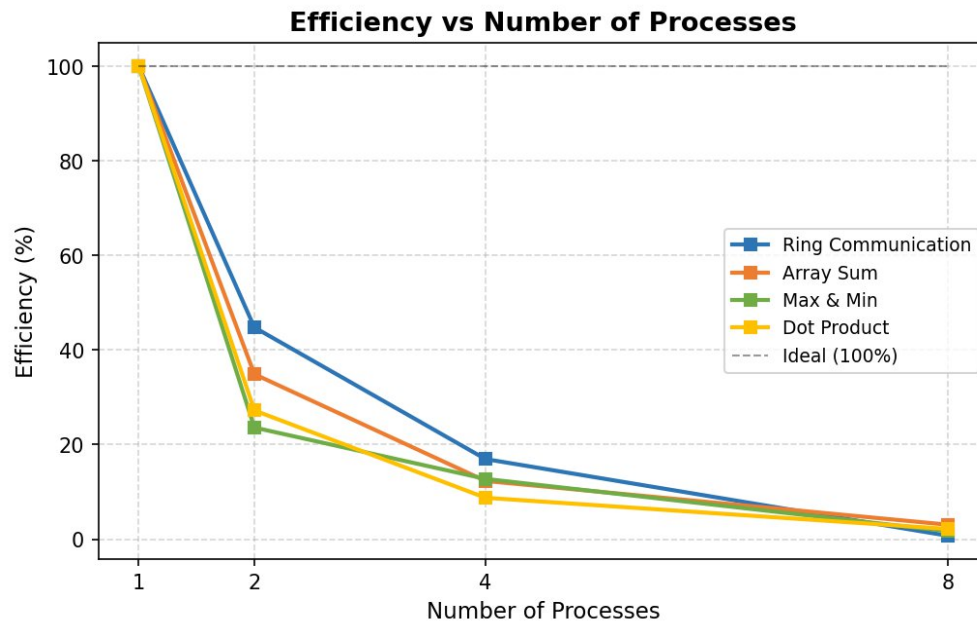
Execution Time vs Number of Processes



Speedup vs Number of Processes



Efficiency vs Number of Processes



Summary Table — Execution Times (seconds)

Exercise	1 Process	2 Processes	4 Processes	8 Processes
Ring Communication	0.000103	0.000115	0.000152	0.001744
Array Sum	0.000067	0.000096	0.000136	0.000273
Max & Min	0.000051	0.000108	0.000100	0.000377
Dot Product	0.000055	0.000101	0.000157	0.000317

Conclusion

These four exercises provided hands-on experience with core MPI programming patterns — point-to-point communication via ring topology, collective distribution and reduction for array sum and dot product, and advanced reduction using `MPI_MAXLOC` and `MPI_MINLOC`.

A consistent observation across all exercises is that for small problem sizes, MPI communication overhead exceeds the computational savings from parallelism, resulting in speedup values below 1.0. This highlights a fundamental principle: MPI is most effective when the computation-to-communication ratio is high, as in large-scale simulations or matrix operations. Increasing problem size would allow parallelism to deliver meaningful speedup.